



Optional类型

北京理工大学计算机学院
金旭亮

讨厌的NullPointerException



由于Stream API中经常需要“级联”多个操作，因此，如果中间某个操作遇到null引用时，如何处理它是个麻烦事。



默认情况下，如果针对null引用调用某个类的实例方法，JVM会抛出NullPointerException，如果不做妥善处理，这个异常可能会“打断”流处理管线。



为了解决这个问题，Java 8专门引入了一个Optional<T>类。

Optional类型简介



Optional对象是一个“可能为null”的对象。



所有那些有可能返回一个null引用的操作，都应该返回一个Optional对象。



Optional对象定义了一个 **isPresent()** 方法用于确定它是否包含一个有效的值。如果值有效，可以调用它的 **get()** 方法提取出这个值。



如果 **isPresent()** 返回 **false**，代码又尝试调用它的 **get()** 方法，JVM会抛出一个 **NoSuchElementException**。

@ ValueBased

Optional<T>

m	empty()	Optional<T>
m	equals(Object?)	boolean
m	filter(Predicate<? super T>)	Optional<T>
m	flatMap(Function<? super T, ? extends Optional<? extends U>>)	Optional<U>
m	get()	T
m	hashCode()	int
m	ifPresent(Consumer<? super T>)	void
m	ifPresentOrElse(Consumer<? super T>, Runnable)	void
m	isEmpty()	boolean
m	isPresent()	boolean
m	map(Function<? super T, ? extends U>)	Optional<U>
m	of(T)	Optional<T>
m	ofNullable(T?)	Optional<T>
m	or(Supplier<? extends Optional<? extends T>>)	Optional<T>
m	orElse(T)	T
m	orElseGet(Supplier<? extends T>)	T
m	orElseThrow()	T
m	orElseThrow(Supplier<? extends X>)	T
m	stream()	Stream<T>
m	toString()	String

构建Optional对象实例



最简单的构建Optional对象的方法是使用其of()方法，另一个empty()方法用于构建一个“空的”Optional对象。

//截取指定长度的字符串

```
public static Optional<String> processString(String string, int length) {  
    if (string == null || length <= 0 || string.length() < length) {  
        return Optional.empty();  
    }  
    //构建Optional对象  
    return Optional.of(string.substring(0, length));  
}
```

示例: CreateOptionalDemo.java



另一个有用的方法是 Optional.ofNullable(obj)。当obj==null时，此方法返回Optional.empty()，否则，返回Optional.of(obj)。

使用Optional对象

返回Optional对象的函数，可以直接用于map函数中：

```
public class CreateOptionalDemo {  
    public static void main(String[] args) {  
        Stream<String> stream = Stream.of("Optional", "type", null, "", "is", "very", "useful");  
        stream.map(str -> processString(str, length: 2)) //截取两个字符  
                .filter(str -> str.isPresent()) //过滤掉空引用  
                .forEach(System.out::println); //输出结果  
    }  
}
```

有不少Stream API函数直接返回Optional对象，以下是一个示例：

```
public class UseOptional {  
    public static void main(String[] args) {  
        var numbers : List<Integer> = List.of(106, 234, 39, 134, 19, 80);  
        //min函数 (还有max函数) , 返回Optional  
        var result : Optional<Integer> = numbers.stream()  
            .min(Comparator.naturalOrder());  
        System.out.println(result.get()); //19  
  
        //of函数返回Optional  
        Optional<String> str = Optional.of("Hello");  
        //isPresent()可以简写为ifPresent  
        str.ifPresent(System.out::println);  
  
        //map函数返回Optional  
        var stringLength : Optional<Integer> = str.map(String::length);  
        stringLength.ifPresent(System.out::println);  
    }  
}
```

针对原始数据类型的Optional



一般来说，只有引用类型才有null值，原始数据类型的变量，比如int，一定义就有值了，不会出现null情况。



但原始数据类型的包装类，比如Integer，使用它们定义的变量，则是有可能为null的。为此，JDK中为它们提供了专门的Optional类型，OptionalInt、OptionalDouble 和 OptionalLong，以避免发生不必要的装箱和拆箱操作。

针对原始数据类型的Optional类型

针对原始数据类型，JDK提供了OptionalInt，OptionalLong和OptionalDouble三个类，拥有getAsXXX()的方法，下面以OptionalInt为例介绍其用法：

```
// 创建一个 OptionalInt实例, 保存数值287
OptionalInt number = OptionalInt.of(287);
//从OptionalInt实例中提取数据的方法
if (number.isPresent()) {
    int value = number.getAsInt();
    System.out.println("Number is " + value);
} else {
    System.out.println("Number is absent.");
}
```

```
// 一些Stream API方法, 返回OptionalInt
OptionalInt numbers = IntStream.of(1, 10, 37, 20, 31)
    .filter(n -> n % 2 == 1).max();
if (numbers.isPresent()) {
    int value = numbers.getAsInt();
    System.out.println("最大值为: " + value);
} else {
    System.out.println("流为空");
}
```

示例: OptionalIntTest.java

返回Optional对象的函数典型编程模式

//用Optional封装有可能为null值的方法

```
private static Optional<OrderClient> getOrderClient() {  
    var ranValue :int = new Random().nextInt( bound: 100);  
    if (ranValue % 2 == 0) {  
        return Optional.empty();  
    } else {  
        var client = new OrderClient(ranValue,  
            name: "client" + ranValue);  
        return Optional.of(client);  
    }  
}
```

OptionalIntTest.java

指定默认值

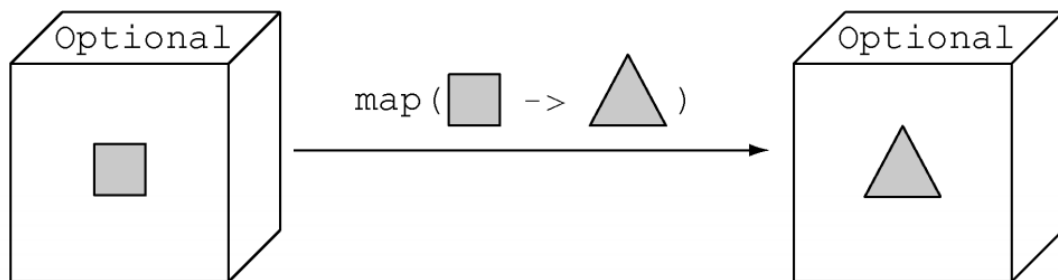
```
var client : Optional<OrderClient> = getOrderClient();  
//client有可能为null,使用orElse指定默认值  
String name = client.map(OrderClient::getName).orElse( other: "无名氏");  
System.out.println(name);
```



Optional类型还有一个orElseThrow()方法，当值为null时，可以抛出指定的异常。请自己编写代码测试一下这个方法。

类型转换

Optional类中定义有一个map方法，可以对结果进行类型转换，得到另一个Optional对象：



```
Optional<String> str = Optional.of("Hello");  
//isPresent()可以简写为ifPresent  
str.ifPresent(System.out::println);  
//将字符串数据转换为Integer  
var stringLength : Optional<Integer> = str.map(String::length);  
System.out.println(stringLength.get());
```

Optional类中还定义有一个flatMap()方法，用得不多，感兴趣的同学请自行探索。